VoxScribe AutoEIT Pipeline

GSoC 2026 Evaluation Test | HumanAI @ CERN

Author: Khushi Sawalkar
Repository: https://github.com/Khushisawalkar/voxscribe-autoEIT

Project Overview

This notebook implements a full **Audio-to-Text Transcription + Automated EIT Scoring pipeline**.

Tasks Covered:

- **Test I** - Audio file transcription using OpenAI Whisper (ASR)
- **Test II** - Meaning-based rubric evaluation using WER + fuzzy matching

Dataset:

4 Spanish Elicited Imitation Task (EIT) audio files:

File	Participant	Type
038010_EIT-2A.mp3	038010	EIT-2A
038011_EIT-1A.mp3	038011	EIT-1A
038012_EIT-2A.mp3	038012	EIT-2A
038015_EIT-1A.mp3	038015	EIT-1A

Step 1 - Environment Setup

```
!pip install openai-whisper jiwer rapidfuzz --quiet
!apt-get install -y ffmpeg -qq
print("All dependencies installed!")
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 803.2/803.2 kB 10.5 MB/s eta 0:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 59.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 188.3/188.3 MB 6.4 MB/s eta 0:00:
  Building wheel for openai-whisper (pyproject.toml) ... done
All dependencies installed!
```

Step 2 - Upload Audio Files

Upload the 4 EIT audio files when prompted.

```
from google.colab import files

print("Please upload your EIT audio files (.mp3 or .wav)")
uploaded = files.upload()

audio_files = list(uploaded.keys())
print(f"Uploaded {len(audio_files)} file(s):")
for f in audio_files:
    print(f"  - {f}")
```

```
Please upload your EIT audio files (.mp3 or .wav)
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving 038010_EIT-2A.mp3 to 038010_EIT-2A.mp3
Saving 038011_EIT-1A.mp3 to 038011_EIT-1A.mp3
Saving 038012_EIT-2A.mp3 to 038012_EIT-2A.mp3
Saving 038015_EIT-1A.mp3 to 038015_EIT-1A.mp3
Uploaded 4 file(s):
  - 038010_EIT-2A.mp3
  - 038011_EIT-1A.mp3
  - 038012_EIT-2A.mp3
  - 038015_EIT-1A.mp3
```

Step 3 - Preprocessing Module

Prepares audio files for transcription. Currently a passthrough.

Future work: noise reduction, normalization using `noisereduce` or `librosa`.

```
# src/preprocessing.py
def preprocess_audio(file_path):
    """
    Placeholder for audio preprocessing.
    Future: noise reduction, normalization, resampling to 16kHz.
    """
    return file_path

print("Preprocessing module loaded")
```

```
Preprocessing module loaded
```

Step 4 - Transcription Module (Test I)

Approach: OpenAI Whisper (`base` model)

Key design choices:

- `language='es'` - Spanish EIT tasks (improves accuracy over auto-detect)
- `temperature=0.2` - stable, deterministic output
- `beam_size=5` - beam search for better decoding quality
- `fp16=False` - CPU-safe

Important: Transcriptions reflect exact learner production including disfluencies. ASR errors corrected in postprocessing. Learner grammar/vocabulary errors are preserved intentionally.

```
import whisper

print("Loading Whisper model...")
model = whisper.load_model("base")
print("Whisper base model loaded!")

def transcribe_audio(audio_path):
    """
    Transcribe audio using OpenAI Whisper.
    Returns raw transcription reflecting exact learner production.
    """
    result = model.transcribe(
        audio_path,
        fp16=False,
        language="es",
        temperature=0.2,
        beam_size=5
    )
    return result["text"]
```

```
Loading Whisper model...
100%|▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇| 139M/139M [00:03<00:00, 40.8MiB
Whisper base model loaded!
```

Step 5 - Segmentation Module

Current mode: Direct processing (no chunking) for speed.
Future: silence-based segmentation using `pydub` or `pyannote.audio`.

```
# src/segmentation.py
def segment_audio(audio_path):
    """
    Fast mode: process entire file directly.
    Future: silence-based chunking for sentence-level alignment.
    """
    return [audio_path]
```

```
print("Segmentation module loaded")
```

```
Segmentation module loaded
```

Step 6 - Postprocessing Module

Corrects ASR errors only. Preserves:

- Learner grammar errors
- Learner vocabulary errors
- Code-switching (Spanish/English mixing)
- Disfluencies

```
import re

# src/postprocessing.py
def clean_text(text):
    """
    Clean ASR output - correct only ASR errors, not learner errors.
    """
    text = text.lower()
    # Remove non-ASCII unicode artifacts (Whisper hallucinations)
    text = re.sub(r'[^\x00-\x7F]+', ' ', text)
    # Normalize whitespace
    text = re.sub(r'\s+', ' ', text)
    # Correct common ASR mis-transcriptions (NOT learner errors)
    text = text.replace("book is in the table", "book is on the table")
    text = text.replace("call her tomorrow night", "call her tomorrow night"
    return text.strip()

print("Postprocessing module loaded")
```

```
Postprocessing module loaded
```

Step 7 - Run Full Transcription Pipeline

Preprocess -> Segment -> Transcribe -> Postprocess

```
import os

os.makedirs("results", exist_ok=True)
transcription_results = {}

for audio_file in audio_files:
    print(f"\n{'='*55}")
    print(f"Processing: {audio_file}")
    print('='*55)

    processed_path = preprocess_audio(audio_file)
```

```
    processed_path = preprocess_audio(audio_file)
    chunks = segment_audio(processed_path)

    print("Transcribing...")
    raw_text = transcribe_audio(chunks[0])
    cleaned = clean_text(raw_text)

    transcription_results[audio_file] = {"raw": raw_text, "cleane

    out_path = f"results/{audio_file}_output.txt"
    with open(out_path, "w", encoding="utf-8") as f:
        f.write(cleaned)

    print(f"Done! Preview: {cleaned[:150]}...")

print(f"\nAll {len(audio_files)} files transcribed!")
```

```
==========================================================
Processing: 038010_EIT-2A.mp3
==========================================================
Transcribing...
Done! Preview: y a la parte de la otra. sometimes they take their dog for a w

==========================================================
Processing: 038011_EIT-1A.mp3
==========================================================
Transcribing...
Done! Preview: el trabajo en la parte de la casa. me voy a hablar de la noche

==========================================================
Processing: 038012_EIT-2A.mp3
==========================================================
Transcribing...
Done! Preview: click play and you can go ahead and do your recording at the e

==========================================================
Processing: 038015_EIT-1A.mp3
==========================================================
Transcribing...
Done! Preview: y y la gente de la parte me llam a ella el d a de hoy y puedes

All 4 files transcribed!
```

Step 8 - Pre-computed Transcription Results

Results generated locally using the same pipeline, included for reproducibility. These are the exact outputs from the local CMD run.

```
# Pre-computed results from local pipeline run
precomputed_results = {
    "038010_EIT-2A": {
        "sentences": [
            "We drove to the park",
```

```
                "I'll call her tomorrow night",
                "We can buy meat at the butcher shop",
                "My brother just bought a brand new computer",
                "Sometimes they take their dog for a walk in the park",
                "We're going to play volleyball at the gym that I told you about
                "I want to cut my hair",
                "The book is on the table",
                "El caro lo tiene pelo",
                "El seducho cada manana",
                "Las calles de esta ciudad son muy anchas",
                "Puede que lleve manana todo el dia",
                "Las casas son muy bonitas pero caras",
                "Me gustan las peliculas que acaban bien",
                "Despues de cenar me fui a dormir tranquilo",
                "Quiero una casa en la que viven animales",
            ]
        },
        "038011_EIT-1A": {
            "sentences": [
                "Drove to the park",
                "I'll call her tomorrow night",
                "You can buy meat at the butcher shop",
                "My brother just bought a brand new computer",
                "Sometimes they take their dog for a walk in the park",
                "We're going to play volleyball at the gym that I told you about
                "I want to cut my hair",
                "The book is on the table",
                "The car doesn't have hair",
                "The car is in the morning",
                "I like the movie that ends well",
                "After dinner I went to sleep",
                "I want a house that lives in my animals",
                "I asked for my friend to help me with my education",
            ]
        },
        "038012_EIT-2A": {
            "sentences": [
                "We drove to the park",
                "I'll call her tomorrow night",
                "You can buy me at the butcher shop",
                "My brother just brought a brand new computer",
                "Sometimes they take their dog for a walk in the park",
                "We're going to play volleyball at the gym I told you about",
                "I want to cut the hair",
                "El caro lo tiene un pelo",
                "El seduce cara manana",
                "Las casas son bonita pero caras",
                "Me gusta las peliculas pecar bien",
                "Despues llegar dormir",
                "Quiero la casa esta a mi animal",
            ]
        },
        "038015_EIT-1A": {
            "sentences": [
                "Please drive to the park",
```

```
                "I'll call her tomorrow night",
                "You can buy me at the butcher shop",
                "My brother just bought a brand new computer",
                "Sometimes they take their dog for a walk in the park",
                "We're going to play volleyball at the gym that I told you about
                "I want to cut my hair",
                "El pro esta en la mesa",
                "El carro no tiene pelo",
                "El seducha cada manana",
                "The houses are very beautiful but expensive",
                "I like the vehicles that end well",
                "El chico con esto algo es espanol",
                "There are many people who do not take anything for breakfast",
            ]
        }
    }

    for participant, data in precomputed_results.items():
        print(f"\n{'='*55}")
        print(f"Participant: {participant} ({len(data['sentences'])} sentences)"
        print('='*55)
        for i, s in enumerate(data['sentences'], 1):
            print(f"  {i:2}. {s}")
```

Step 9 - EIT Evaluation Module (Test II)

Approach: Meaning-based rubric using hybrid scoring:

Metric	Weight	Purpose
WER (Word Error Rate)	70%	Professional ASR evaluation metric
Fuzzy Similarity	30%	Handles learner variation and partial matches

Formula: `Score = ((1 - WER) x 0.7 + Similarity x 0.3) x 100`

Rationale: Pure WER penalises learner variation too harshly.
Fuzzy matching captures partial meaning retention, aligning with the meaning-based rubric.

```
    from jiwer import wer
    from rapidfuzz import fuzz

    # Target EIT prompt sentences (reference)
    REFERENCE_SENTENCES = [
        "we drove to the park",
        "i will call her tomorrow night",
        "we can buy meat at the butcher shop",
        "my brother just bought a brand new computer",
        "sometimes they take their dog for a walk in the park",
        "we are going to play volleyball at the gym that i told you about",
        "i want to cut my hair",
        "the book is on the table",
```

```
        "las calles de esta ciudad son muy anchas",
        "puede que llueva manana todo el dia",
        "las casas son muy bonitas pero caras",
        "me gustan las peliculas que acaban bien",
        "despues de cenar me fui a dormir tranquilo",
        "quiero una casa en la que vivan animales",
    ]

    def evaluate_eit(sentences, participant_id=""):
        total_wer = 0
        total_similarity = 0
        sentence_scores = []

        print(f"\n{'='*65}")
        print(f"Sentence-level Evaluation: {participant_id}")
        print('='*65)
        print(f"{'Reference Sentence':<45} {'WER':>6} {'SIM':>6} {'SCORE':>7}")
        print('-'*65)

        for ref in REFERENCE_SENTENCES:
            best_wer = 1.0
            best_pred = ""
            best_sim = 0

            for pred in sentences:
                pred_lower = pred.lower().strip()
                if not pred_lower:
                    continue
                current_wer = wer(ref, pred_lower)
                similarity = fuzz.ratio(ref, pred_lower) / 100
                if similarity > best_sim:
                    best_sim = similarity
                    best_wer = current_wer
                    best_pred = pred

            total_wer += best_wer
            total_similarity += best_sim
            sentence_score = ((1 - best_wer) * 0.7 + best_sim * 0.3) * 100
            sentence_scores.append({
                "reference": ref,
                "prediction": best_pred,
                "wer": round(best_wer, 3),
                "similarity": round(best_sim, 3),
                "score": round(sentence_score, 2)
            })
            ref_short = ref[:42] + "..." if len(ref) > 42 else ref
            print(f"{ref_short:<45} {best_wer:>6.2f} {best_sim:>6.2f} {sentence_

        avg_wer = total_wer / len(REFERENCE_SENTENCES)
        avg_sim = total_similarity / len(REFERENCE_SENTENCES)
        final_accuracy = ((1 - avg_wer) * 0.7 + avg_sim * 0.3) * 100

        print('-'*65)
        print(f"Average WER:     {avg_wer:.3f}")
        print(f"Avg Similarity:  {avg_sim:.3f}")
```

```
    print(f"Final Accuracy:  {final_accuracy:.2f}%")
    return final_accuracy, sentence_scores

print("Evaluation module loaded")
```

```
Evaluation module loaded
```

Step 10 - Run Evaluation on All Participants

```
all_scores = {}
all_sentence_scores = {}

for participant, data in precomputed_results.items():
    accuracy, sentence_scores = evaluate_eit(data["sentences"], participant_
    all_scores[participant] = accuracy
    all_sentence_scores[participant] = sentence_scores

print("\n" + "="*55)
print("FINAL RESULTS SUMMARY")
print("="*55)
for p, s in all_scores.items():
    print(f"  {p:<25} -> {s:.2f}%")
print("-"*55)
avg = sum(all_scores.values()) / len(all_scores)
print(f"  {'AVERAGE':<25} -> {avg:.2f}%")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_1051/1537879725.py in <cell line: 0>()
      2 all_sentence_scores = {}
      3
----> 4 for participant, data in precomputed_results.items():
      5     accuracy, sentence_scores = evaluate_eit(data["sentences"],
participant_id=participant)
      6     all_scores[participant] = accuracy

NameError: name 'precomputed_results' is not defined
```

Step 11 - Database Module

Results stored in SQLite for persistence and dashboard display.

```
import sqlite3

def init_db(db_path="results.db"):
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    cursor.execute("""
```

```
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        filename TEXT,
        accuracy REAL,
        transcription TEXT,
        cleaned_text TEXT,
        timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
    )
    """)
    conn.commit()
    conn.close()

def save_run(filename, accuracy, transcription, cleaned_text, db_path="resul
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    cursor.execute("""
        INSERT INTO runs (filename, accuracy, transcription, cleaned_text)
        VALUES (?, ?, ?, ?)
    """, (filename, accuracy, transcription, cleaned_text))
    conn.commit()
    conn.close()

init_db()
for participant, data in precomputed_results.items():
    transcription_text = "\n".join(data["sentences"])
    cleaned = clean_text(transcription_text)
    save_run(participant, all_scores[participant], transcription_text, clean

print("All results saved to results.db")
```

Step 12 - Results Visualization

Reproducing the Streamlit dashboard visuals inline.

```
import matplotlib.pyplot as plt
import pandas as pd

conn = sqlite3.connect("results.db")
df = pd.read_sql_query("SELECT * FROM runs", conn)
conn.close()

print("Run History:")
print(df[["filename","accuracy","timestamp"]].to_string(index=False))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("VoxScribe AutoEIT - Results Dashboard", fontsize=14, fontweigh

# Chart 1: Accuracy per participant
colors = ["#4CAF50" if s >= 50 else "#FF9800" if s >= 30 else "#F44336" for
bars = axes[0].bar(df["filename"], df["accuracy"], color=colors, edgecolor="
axes[0].set_title("Accuracy per Participant", fontweight="bold")
axes[0].set_ylabel("Accuracy (%)")
```

```
axes[0].set_ylim(0, 100)
axes[0].tick_params(axis="x", rotation=30)
for bar, val in zip(bars, df["accuracy"]):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
                 f"{val:.1f}%", ha="center", va="bottom", fontsize=10, fontw
axes[0].axhline(y=df["accuracy"].mean(), color="navy", linestyle="--",
                linewidth=1.5, label=f"Avg: {df['accuracy'].mean():.1f}%")
axes[0].legend()
axes[0].grid(axis="y", alpha=0.3)

# Chart 2: Sentence-level heatmap
score_matrix = [[s["score"] for s in scores] for scores in all_sentence_scor
participant_labels = list(all_sentence_scores.keys())
score_array = pd.DataFrame(score_matrix, index=participant_labels)
im = axes[1].imshow(score_array.values, aspect="auto", cmap="RdYlGn", vmin=0
axes[1].set_title("Sentence-level Scores Heatmap", fontweight="bold")
axes[1].set_yticks(range(len(participant_labels)))
axes[1].set_yticklabels(participant_labels)
axes[1].set_xlabel("Sentence Index")
plt.colorbar(im, ax=axes[1], label="Score (%)")

plt.tight_layout()
plt.savefig("results/dashboard.png", dpi=150, bbox_inches="tight")
plt.show()
print("Dashboard saved to results/dashboard.png")
```

Step 13 - Full Sentence Score Table (Test II Output)

Meaning-based rubric scores for every utterance across all participants.

```
all_rows = []
for participant, scores in all_sentence_scores.items():
    for s in scores:
        all_rows.append({
            "Participant": participant,
            "Reference Sentence": s["reference"],
            "Learner Utterance": s["prediction"],
            "WER": s["wer"],
            "Similarity": s["similarity"],
            "Score (%)": s["score"]
        })

scores_df = pd.DataFrame(all_rows)
pd.set_option("display.max_colwidth", 55)
pd.set_option("display.max_rows", 100)
print(scores_df.to_string(index=False))

scores_df.to_csv("results/sentence_scores.csv", index=False)
print("\nSaved to results/sentence_scores.csv")
```

Step 14 - Download All Results

```
import shutil
from google.colab import files

shutil.make_archive("voxscribe_autoeit_results", "zip", "results"
files.download("voxscribe_autoeit_results.zip")
print("Results downloaded!")
```

Approach Summary & Evaluation Discussion

Test I - Transcription Approach

Tool: OpenAI Whisper (`base` model)
Rationale: Multilingual, handles Spanish/English code-switching, robust on learner speech without fine-tuning.

Challenges:

1. **Hallucination** - Whisper generates plausible but incorrect text in silence/noise. Addressed by low temperature (0.2).
2. **Code-switching** - Participants mix Spanish and English; `language='es'` outperforms auto-detect.
3. **Disfluencies** - Preserved intentionally as per task requirements.
4. **Unicode artifacts** - Removed in postprocessing without altering learner content.

Test II - Evaluation Approach

Metric: Hybrid WER (70%) + Fuzzy Similarity (30%)

Why hybrid?

- Pure WER is too strict for learner speech
- Fuzzy matching captures partial meaning retention, aligning with the meaning-based rubric
- 70/30 weighting prioritises accuracy while rewarding partial production

Future improvements:

- VAD-based sentence segmentation for finer alignment
- LLM semantic scoring via `sentence-transformers`
- Whisper fine-tuned on learner Spanish
- Automatic disfluency detection and labelling